IPv6 지원 및 듀얼스택 분석

개요

rqbit은 IPv4와 IPv6를 모두 지원하는 듀얼스택 BitTorrent 클라이언트로 구현되어 있습니다. 이는 현대적인 네트워크 환경에서 IPv6 전환을 지원하면서도 기존 IPv4 네트워크와의 호환성을 유지하기 위한 설계입니다. 핵심적으로 `librqbit-dualstack-sockets` 외부 크레이트를 활용하여 통합된 소켓 관리를 제공합니다.

핵심 IPv6 지원 구성 요소

1. Compact IP 주소 직렬화

CompactSerialize 트레이트 구현

```
pub  trait CompactSerialize: Sized {
type Slice: AsRef<[u8]>;
fn expecting() -> &'static str;
fn as_slice(&self) -> Self::Slice;
fn from_slice(buf: &[u8]) -> Option<Self>;
}
```

```
// IPv4 주소 (4바이트)
impl CompactSerialize for Ipv4Addr {
type Slice = [u8; 4];
```

```
fn as_slice(&self) -> Self::Slice {
self.octets()
}
fn from_slice(buf: &[u8]) -> Option<Self> {
Some(Self::from(TryInto::<[u8; 4]>::try_into(buf.get(..4)?).unwrap()))
}
}

// IPv6 주소 (16바이트)
impl CompactSerialize for Ipv6Addr {
type Slice = [u8; 16];
fn as_slice(&self) -> Self::Slice {
self.octets()
}
fn from_slice(buf: &[u8]) -> Option<Self> {
Some(Self::from(TryInto::<[u8; 16]>::try_into(buf.get(..16)?).unwrap()))
}
}
```

가변 길이 IP 주소 처리

```
impl CompactSerialize for IpAddr {
type Slice = SmallSlice<16>; // 최대 16바이트 (IPv6)
```

```
fn as_slice(&self) -> Self::Slice {
match self {
IpAddr::V4(a) => SmallSlice::new_from_buf(&a.as_slice()), // 4바이트
IpAddr::V6(a) => SmallSlice::new_from_buf(&a.as_slice()), // 16바이트
}
}
fn from_slice(buf: &[u8]) -> Option<Self> {
match buf.len() {
4 => Some(Ipv4Addr::from_slice_unchecked_len(buf).into()),
16 => Some(Ipv6Addr::from_slice_unchecked_len(buf).into()),
_ => None,
}
}
}
```

2. 소켓 주소 직렬화

IPv4 소켓 주소 (6바이트: IP 4바이트 + 포트 2바이트)

```
impl CompactSerialize for SocketAddrV4 {
type Slice = [u8; 6];
fn as_slice(&self) -> Self::Slice {
let mut data = [0u8; 6];
data[..4].copy_from_slice(&self.ip().octets());
```

```
data[4..6].copy_from_slice(&self.port().to_be_bytes());
data
}
fn from_slice_unchecked_len(buf: &[u8]) -> Self {
let ip = Ipv4Addr::from_slice_unchecked_len(&buf[..4]);
let port = u16::from_be_bytes([buf[4], buf[5]]);
SocketAddrV4::new(ip, port)
}
}
```

IPv6 소켓 주소 (18바이트: IP 16바이트 + 포트 2바이트)

```
impl CompactSerialize for SocketAddrV6 {
type Slice = [u8; 18];
fn as_slice(&self) -> Self::Slice {
let mut data = [0u8; 18];
data[..16].copy_from_slice(&self.ip().octets());
data[16..18].copy_from_slice(&self.port().to_be_bytes());
data
}
fn from_slice_unchecked_len(buf: &[u8]) -> Self {
let ip = Ipv6Addr::from_slice_unchecked_len(&buf[..16]);
let port = u16::from_be_bytes([buf[16], buf[17]]);
SocketAddrV6::new(ip, port, 0, 0) // flow_info=0, scope_id=0
```

```
}

}
```

가변 길이 소켓 주소 처리

```
impl CompactSerialize for SocketAddr {

type Slice = SmallSlice<18>; // 최대 18바이트 (IPv6)

fn as_slice(&self) -> Self::Slice {

let mut s = SmallSlice::new_from_buf(self.ip().as_slice().as_ref());

s.extend(&self.port().to_be_bytes());

s

}

fn from_slice(buf: &[u8]) -> Option<Self> {

match buf.len() {

6 => Some(SocketAddrV4::from_slice_unchecked_len(buf).into()), // IPv4

18 => Some(SocketAddrV6::from_slice_unchecked_len(buf).into()), // IPv6

_ => None,

}

}

}
```

듀얼스택 네트워킹 구현

1. 리스너 설정

```
// 기본 듀얼스택 바인딩
pub struct ListenerOptions {
pub mode: ListenerMode,
pub listen_addr: SocketAddr, // 기본값: [::]:0 (IPv6 UNSPECIFIED)
pub enable_upnp_port_forwarding: bool,
pub utp_opts: Option<UtpListenerOptions>,
}

impl Default for ListenerOptions {
fn default() -> Self {
Self {
mode: ListenerMode::TcpOnly,
listen_addr: (Ipv6Addr::UNSPECIFIED, 0).into(), // 듀얼스택 바인딩
enable_upnp_port_forwarding: false,
utp_opts: None,
}
}
}
```

2. 연결 통계 및 구분

```
// 연결 타입별 통계 수집

impl StreamConnector {

async fn with_stat<T>(

&self,

kind: ConnectionKind,

is_ipv6: bool, // IPv6 여부로 통계 구분

fut: impl Future<Output = anyhow::Result<T>>,

) -> anyhow::Result<T> {

// IPv4/IPv6별 연결 통계 수집

let result = fut.await;

// 성공/실패 통계 업데이트

result

}

}
```

3. 프로토콜별 연결 처리

```
// TCP 연결

async fn tcp_connect(&self, addr: SocketAddr) -> anyhow::Result<(TcpReadHalf, Tc

self.with_stat(

ConnectionKind::Tcp,

addr.is_ipv6(), // IPv6 여부 확인

librqbit_dualstack_sockets::tcp_connect(addr, self.bind_device.as_ref()),
```

```
).await
}

// SOCKS 프록시 연결
if let Some(proxy) = self.proxy_config.as_ref() {
let (r, w) = self.with_stat(
ConnectionKind::Socks,
addr.is_ipv6(), // IPv6 여부 확인
proxy.connect(addr)
).await?;
}

// uTP 연결
let conn = self.with_stat(
ConnectionKind::Utp,
addr.is_ipv6(), // IPv6 여부 확인
sock.connect(addr)
).await?;
```

멀티캐스트 및 서비스 검색

1. LSD (Local Service Discovery) 듀얼스택

```
const LSD_PORT: u16 = 6771;

const LSD_IPV4: SocketAddrV4 = SocketAddrV4::new(Ipv4Addr::new(239, 192, 152, 14

const LSD_IPV6: SocketAddrV6 = SocketAddrV6::new(

Ipv6Addr::new(0xff15, 0, 0, 0, 0, 0, 0xefc0, 0x988f), // 사이트 로컬 멀티캐스트

LSD_PORT,

0,

0,

);



// 듀얼스택 멀티캐스트 소켓 생성

let socket = MulticastUdpSocket::new(

(Ipv6Addr::UNSPECIFIED, LSD_PORT).into(), // IPv6 바인딩 (듀얼스택)

LSD_IPV4, // IPv4 멀티캐스트 그룹

LSD_IPV6, // IPv6 멀티캐스트 그룹

None, // IPv6 링크 로컬 (미사용)

opts.bind_device, // 특정 인터페이스 바인딩

).await?;
```

2. UPnP SSDP 듀얼스택

```
const SSDP_PORT: u16 = 1900;

const SSDP_MCAST_IPV4: SocketAddrV4 =
```

```
SocketAddrV4::new(Ipv4Addr::new(239, 255, 255, 250), SSDP_PORT);

const SSDP_MCAST_IPV6_SITE_LOCAL: SocketAddrV6 = SocketAddrV6::new(

Ipv6Addr::new(0xff05, 0, 0, 0, 0, 0, 0, 0xc), // 사이트 로컬 멀티캐스트

SSDP_PORT,

0,

0,

);



// 듀얼스택 SSDP 소켓

let socket = MulticastUdpSocket::new(

(Ipv6Addr::UNSPECIFIED, SSDP_PORT).into(),

SSDP_MCAST_IPV4,

SSDP_MCAST_IPV6_SITE_LOCAL,

None, // 링크 로컬 미사용 (VLC 호환성 문제)

None,

).await?;
```

트래커 통신에서의 IPv6 지원

1. UDP 트래커 IPv6 지원

```
impl Response {

pub fn parse(buf: &[u8], is_ipv6: bool) -> anyhow::Result<(TransactionId, Self)>
```

```
// IPv6 여부에 따른 응답 파싱
let response = match Self::parse_response(action, is_ipv6, buf) {
Ok(r) => r,
Err(e) => return Err(e),
};
Ok((tid, response))
}


fn parse_response(action: u32, is_ipv6: bool, mut buf: &[u8]) -> anyhow::Result<
match action {
ACTION_ANNOUNCE => {
let mut addrs = Vec::new();
while !buf.is_empty() {
let (addr, b2) = if is_ipv6 {
// IPv6 주소 파싱 (16바이트 + 2바이트 포트)
let (ip, b2) = u128::parse_num(buf)?;
(IpAddr::V6(Ipv6Addr::from(ip)), b2)
} else {
// IPv4 주소 파싱 (4바이트 + 2바이트 포트)
let (ip, b2) = u32::parse_num(buf)?;
(IpAddr::V4(Ipv4Addr::from(ip)), b2)
};
let (port, b2) = u16::parse_num(b2)?;
```

```
addrs.push(SocketAddr::new(addr, port));
buf = b2;
}
Ok(Response::Announce { addrs })
}
}
}
}
```

2. UDP 트래커 클라이언트 듀얼스택 바인딩

```
impl UdpTrackerClient {
pub async fn new(bind_device: Option<&BindDevice>) -> anyhow::Result<Self> {
// IPv6 UNSPECIFIED로 바인딩하여 듀얼스택 지원
let addr = SocketAddr::new(IpAddr::V6(Ipv6Addr::UNSPECIFIED), 0);
let sock = UdpSocket::bind_udp(
addr,
librqbit_dualstack_sockets::BindOpts {
device: bind_device,
..Default::default()
},
)?;
Ok(Self { sock, transactions: Default::default() })
```

```
}

}
```

3. 트래커 주소 해결

```
// 도메인 이름 해결 시 IPv4/IPv6 모두 고려

let res = match host {

url::Host::Domain(name) => {

// 첫 번째 IPv4와 첫 번째 IPv6 주소만 사용

let mut v4: Option<SocketAddrV4> = None;

let mut v6: Option<SocketAddrV6> = None;

for addr in lookup_host((name, port)).await? {

match addr {

SocketAddr::V4(a) if v4.is_none() => v4 = Some(a),

SocketAddr::V6(a) if v6.is_none() => v6 = Some(a),

_ => {}

}

}

match (v4, v6) {

(Some(v4), Some(v6)) => UdpTrackerResolveResult::Both(v4, v6),

(Some(v4), None) => UdpTrackerResolveResult::One(v4.into()),

(None, Some(v6)) => UdpTrackerResolveResult::One(v6.into()),

(None, None) => anyhow::bail!("no addresses resolved"),
```

```
}

}

url::Host::Ipv4(addr) => UdpTrackerResolveResult::One((addr, port).into()),

url::Host::Ipv6(addr) => UdpTrackerResolveResult::One((addr, port).into()),

};
```

피어 교환에서의 IPv6 지원

1. Extended Handshake IPv6 필드

```
#[derive(Serialize, Deserialize, Debug, Clone)]

pub struct ExtendedHandshake {

#[serde(skip_serializing_if = "Option::is_none")]

pub yourip: Option<CompactIpAddr>, // 상대방 IP (IPv4/IPv6 자동 감지)

#[serde(skip_serializing_if = "Option::is_none")]

pub ipv6: Option<CompactIpV6>, // 자신의 IPv6 주소

#[serde(skip_serializing_if = "Option::is_none")]

pub ipv4: Option<CompactIpV4>, // 자신의 IPv4 주소

}


impl ExtendedHandshake {

pub fn external_ip(&self) -> Option<IpAddr> {

// yourip 우선, 없으면 ipv4, 마지막으로 ipv6
```

```
if let Some(ref b) = self.yourip {
return Some(b.0.into());
}
if let Some(ref b) = self.ipv4 {
return Some(b.0.into());
}
if let Some(ref b) = self.ipv6 {
return Some(b.0.into());
}
None
}
}
```

2. PEX (Peer Exchange) IPv6 지원

```
// ut_pex 확장에서 IPv6 피어 교환 지원
// IPv4와 IPv6 피어 목록을 별도로 관리하고 교환
```

블록리스트에서의 IPv6 지원

1. 듀얼스택 블록리스트

```
pub struct Blocklist {
v4: IntervalTreeWithSize<Ipv4Addr>, // IPv4 범위 트리
```

```
v6: IntervalTreeWithSize<Ipv6Addr>, // IPv6 범위 트리
}

impl Blocklist {
pub fn new(
v4_ranges: impl IntoIterator<Item = Range<Ipv4Addr>>,
v6_ranges: impl IntoIterator<Item = Range<Ipv6Addr>>,
) -> Self {
Self {
v4: IntervalTreeWithSize::new(v4_ranges),
v6: IntervalTreeWithSize::new(v6_ranges),
}
}

pub fn is_blocked(&self, addr: IpAddr) -> bool {
match addr {
IpAddr::V4(addr) => self.v4.contains_point(addr),
IpAddr::V6(addr) => self.v6.contains_point(addr),
}
}
}
```

2. 블록리스트 파일 파싱

```rust
fn parse_ip_range_line(line: &str) -> Option<IpRange> {
let is_ipv6 = line.matches(":").count() > 2; // IPv6 감지
let (start_str, end_str) = if is_ipv6 {
line.split_once(':')? // IPv6 범위 파싱
} else {
line.split_once('-')? // IPv4 범위 파싱
};
let start: IpAddr = start_str.parse().ok()?;
let end: IpAddr = end_str.parse().ok()?;
match (start, end) {
(IpAddr::V4(start), IpAddr::V4(end)) => {
let end = Ipv4Addr::from_bits(end.to_bits().saturating_add(1));
Some(IpRange::V4(start..end))
}
(IpAddr::V6(start), IpAddr::V6(end)) => {
let end = Ipv6Addr::from_bits(end.to_bits().saturating_add(1));
Some(IpRange::V6(start..end))
}
_ => None, // 혼합 타입은 지원하지 않음
}
}
```

현재 IPv6 지원 한계

1. UPnP 포트 포워딩 제한

```
// UPnP 포트 포워딩은 현재 IPv4만 지원
pub async fn discover_once(
tx: &UnboundedSender<UpnpDiscoverResponse>,
kind: &str,
timeout: Duration,
bind_device: Option<&BindDevice>,
) -> anyhow::Result<()> {
// TODO: IPv6 지원 필요 - 하드웨어 테스트 환경 부족으로 미구현
let socket = UdpSocket::bind_udp(
(Ipv4Addr::UNSPECIFIED, 0).into(), // IPv4만 바인딩
librqbit_dualstack_sockets::BindOpts {
device: bind_device,
..Default::default()
},
)?;
}
```

2. 링크 로컬 주소 제한

```
// VLC 호환성 문제로 IPv6 링크 로컬 주소 거부
```

```
if matches!(addr.ip(), IpAddr::V6(a) if a.is_unicast_link_local()) {

debug!(?addr, "refusing to reply to a link-local address");

return Ok(None);

}
```

3. DHT IPv6 지원 상태

현재 DHT 구현에서 IPv6 지원 여부는 명시적으로 확인되지 않았으나, 기본적인 소켓 주소 처리는 IPv6를 지원합니다.

성능 및 최적화

1. 메모리 효율성

```
// SmallSlice를 사용한 가변 길이 주소 처리

pub struct SmallSlice<const N: usize> {

data: [u8; N], // 스택 할당 (최대 16바이트)

len: usize, // 실제 사용 길이

}


// IPv4는 4바이트, IPv6는 16바이트만 사용

impl CompactSerialize for IpAddr {

type Slice = SmallSlice<16>; // 최대 크기로 할당하되 실제 사용량은 최소화

}
```

2. 직렬화 최적화

```rust
// 고정 길이 타입은 컴파일 타임 최적화
impl CompactSerializeFixedLen for Ipv4Addr {
fn fixed_len() -> usize { 4 }
}

impl CompactSerializeFixedLen for Ipv6Addr {
fn fixed_len() -> usize { 16 }
}

impl CompactSerializeFixedLen for SocketAddrV4 {
fn fixed_len() -> usize { 6 }
}

impl CompactSerializeFixedLen for SocketAddrV6 {
fn fixed_len() -> usize { 18 }
}
```

3. 네트워크 효율성

- **듀얼스택 소켓**: 하나의 소켓으로 IPv4/IPv6 동시 처리
- **프로토콜별 통계**: IPv4/IPv6 연결 성능을 별도로 모니터링
- **자동 감지**: 주소 길이로 프로토콜 자동 판별

보안 고려사항

1. 주소 검증

```
// 입력 주소 길이 검증
fn from_slice(buf: &[u8]) -> Option<Self> {
match buf.len() {
4 => Some(Ipv4Addr::from_slice_unchecked_len(buf).into()),
16 => Some(Ipv6Addr::from_slice_unchecked_len(buf).into()),
_ => None, // 잘못된 길이는 거부
}
}
```

2. 범위 검증

```
// IPv6 주소 범위 오버플로우 방지
let end = Ipv6Addr::from_bits(end.to_bits().saturating_add(1));
```

3. 프라이버시 주소

IPv6 프라이버시 확장 주소나 임시 주소에 대한 특별한 처리는 현재 구현되지 않았습니다.

향후 개선 방안

1. 완전한 IPv6 지원

- **UPnP IPv6**: IPv6 환경에서의 포트 포워딩 지원
- **링크 로컬**: IPv6 링크 로컬 주소 완전 지원
- **DHT IPv6**: DHT 네트워크에서 IPv6 노드 지원

2. 성능 최적화

- **Happy Eyeballs**: IPv4/IPv6 동시 연결 시도로 연결 속도 향상
- **주소 선택**: RFC 6724 기반 주소 선택 알고리즘
- **멀티홈**: 여러 네트워크 인터페이스 최적 활용

3. 고급 기능

- **IPv6 전용 모드**: IPv4 비활성화 옵션
- **프라이버시 확장**: IPv6 프라이버시 주소 지원
- **모바일 IPv6**: 모바일 환경에서의 IPv6 지원

결론

rqbit의 IPv6 및 듀얼스택 지원은 현대적인 네트워크 환경에 잘 적응된 구현입니다. 핵심적으로 compact 직렬화를 통한 효율적인 주소 처리, 듀얼스택 소켓을 통한 통합된 네트워킹, 그리고 프로토콜별 통계 수집을 제공합니다.

현재 일부 제한사항(UPnP IPv6, 링크 로컬 주소 등)이 있지만, 기본적인 BitTorrent 프로토콜에서는 IPv6를 완전히 지원하며, 향후 확장을 위한 견고한 기반을 제공합니다. 특히 메모리 효율적인 주소 처리와 자동 프로토콜 감지를 통해 성능과 호환성을 모두 확보하고 있습니다.